Sparse Attention as Compact Kernel Regression

Объяснение статьи

Константин Корнилов

10 октября 2026

Основная идея статьи

- Классический softmax self-attention можно рассматривать как ядерную регрессию с гауссовым ядром на инфернсе. Это было установлено в предыдущих работах.
- Softmax присваивает положительные веса всем доступным ключам. На длинных последовательностях множество слабых вкладов может размывать влияние релевантных элементов. Разреженные механизмы внимания позволяют присваивать части ключей строго нулевые веса.
- Авторы связывают разреженные механизмы внимания с ядерной регрессией с компактным носителем. Также предлагается новый метод r-ReLUmax, сохраняющий ключи в заданном диапазоне сходства относительно лучшего ключа.
- Для сравнения ядер используется Memory Mosaics (Zhang et al., ICLR 2025) — архитектура на основе ассоциативной памяти, которая явно реализует ядерную регрессию.

$$\text{Attention}(Q, K, V) = \text{softmax}\boxed{\frac{QK^T}{\sqrt{d_k}}}V$$

Что предложено в статье

- Ряд разреженных механизмов внимания представлен в виде ядерной регрессии
- Представлен новый механизм разреженного внимания r-ReLUmax
- Проведены эксперименты на архитектуре Memory Mosaics, показывающие преимущества ряда разреженных методов по сравнению с классическим softmax attention

Что такое ядерная регрессия

Nadaraya-Watson estimator — непараметрический метод ядерной регрессии. Предсказание — взвешенное среднее наблюдаемых значений; веса определяются близостью входов к запросу.
В обозначениях key, value:

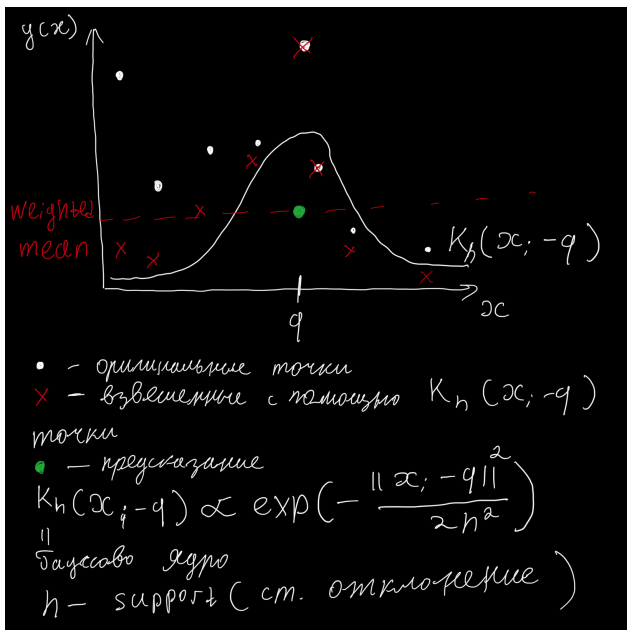
$$\hat{v}_n(q) = \sum_{i=1}^{n-1} \frac{K_h(k_i - q)}{\sum_{j=1}^{n-1} K_h(k_j - q)} \cdot v_i$$

Эта величина оценивает условное ожидание $\mathbb{E}[V_n \mid q]$.
$\{(k_i, v_i)\}_{i=1}^{n-1}$ — обучающая выборка, K_h — ядерная функция с bandwidth (шириной окна) h. $q = k_n$ — входной вектор запроса.

- Часто используется гауссово ядро

$$K_h(u) \propto \exp\left(-\frac{\|u\|^2}{2h^2}\right)$$

- На практике важно подобрать подходящее h



Представление attention как ядерной регрессии

Рассмотрим внимание к предыдущим позициям, как в contextual memory архитектуры Memory Mosaics. Здесь текущая позиция исключена; обычное causal self-attention допускает внимание и к ней. Для вывода предположим, что $\|q\| = \|k_i\| = 1$.

Формула гауссового ядра с точностью до общего множителя:

$$K_h(k_i - q) \propto \exp\left(-\frac{\|k_i - q\|^2}{2h^2}\right) = \exp\left(-\frac{\|k_i\|^2 - 2k_i^T q + \|q\|^2}{2h^2}\right) = \exp\left(\frac{k_i^T q - 1}{h^2}\right)$$

Также получаем результат $\frac{1}{2}\|k_i - q\|^2 = 1 - k_i^T \cdot q$, который будет использоваться в дальнейшем.

Итак, подставив формулу в формулу Nadaraya-Watson estimator, получаем:

$$\hat{v}_n(q) = \sum_{i=1}^{n-1} \frac{\exp\left(\frac{k_i^T \cdot q - 1}{h^2}\right)}{\sum_{j=1}^{n-1} \exp\left(\frac{k_j^T \cdot q - 1}{h^2}\right)} v_i = \sum_{i=1}^{n-1} \frac{\exp\left(\frac{k_i^T q}{h^2}\right)}{\sum_{j=1}^{n-1} \exp\left(\frac{k_j^T q}{h^2}\right)} v_i = \sum_{i=1}^{n-1} \mathrm{softmax}_i\left(\frac{Kq}{h^2}\right) v_i$$

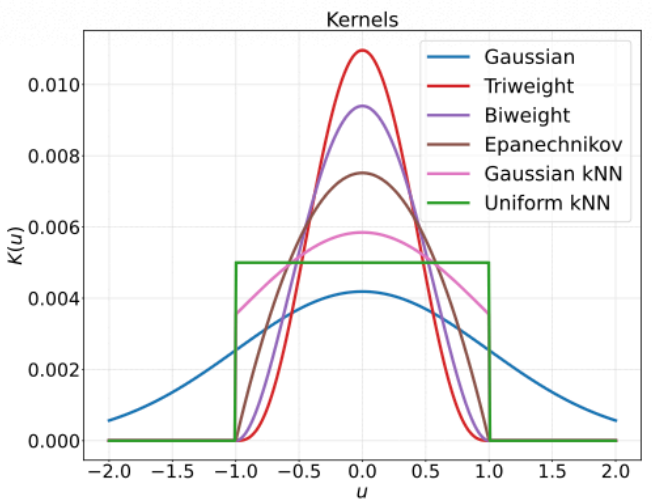
- стандартный нормировочный коэффициент $\sqrt{d_k}$ соответствует $h = d_k^{\frac{1}{4}}$ в этой интерпретации
- в некоторых работах выбирают другой h для softmax attention

Другие часто используемые ядра

Выбор ядра определяет, как расстояние между q и k_i влияет на вес наблюдения.

У гауссовского ядра неограниченный носитель: $K_h(u) > 0$ для любого конечного u. Поэтому каждый доступный ключ получает ненулевой вес.

У ядер с компактным носителем: $K_h(u) = 0$ при $\|u\| > R$, где $R > 0$. Достаточно далёкие ключи не влияют на предсказание.



- Uniform: $1_{\|u\| \le h}$ — одинаковые веса внутри носителя. Соответствует нулевой степени полиномиального семейства ($r = 0$)
- Epanechnikov: $\left[1 - \frac{\|u\|^2}{h^2}\right]_+$ — параболическое убывание
- Biweight: $\left[1 - \frac{\|u\|^2}{h^2}\right]_+^2$ — более гладкое зануление на границе носителя
- Triweight: $\left[1 - \frac{\|u\|^2}{h^2}\right]_+^3$ — ещё более гладкое зануление на границе носителя
- kNN / top-k: $K_h(u) \cdot 1_{i \in N_{k(q)}}$ — оставляем только k ближайших ключей. В качестве K_h можно взять uniform или Gaussian.

$u = k_i - q$, $[z]_+ = \max(0, z)$, $N_{k(q)}$ — множество k ближайших ключей.

Формулы приведены с точностью до нормировочного множителя.

Известные методы sparse attention

В отличие от softmax, разреженные механизмы внимания позволяют присваивать некоторым ключам строго нулевые веса.

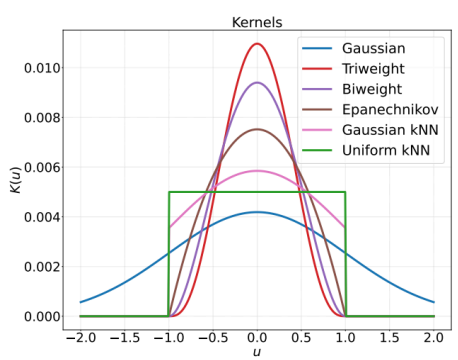
- Top-k softmax — применяет softmax только к k наибольшим значениям сходства; остальные веса равны нулю.
- Normalized ReLU — применяет $[z]_+ = \max(0, z)$ и нормирует полученные веса: $p_i = \frac{[z_i]_+}{\sum_j [z_j]_+}$. При нулевом знаменателе не определён.
- Sparsemax — евклидова проекция вектора сходства на вероятностный симплекс: $p_i = [z_i - \tau]_+$, где τ выбирается так, чтобы $\sum_i p_i = 1$.
- α-entmax — определяет веса внимания через баланс между сходством и энтропией распределения весов. Параметр α управляет разреженностью:

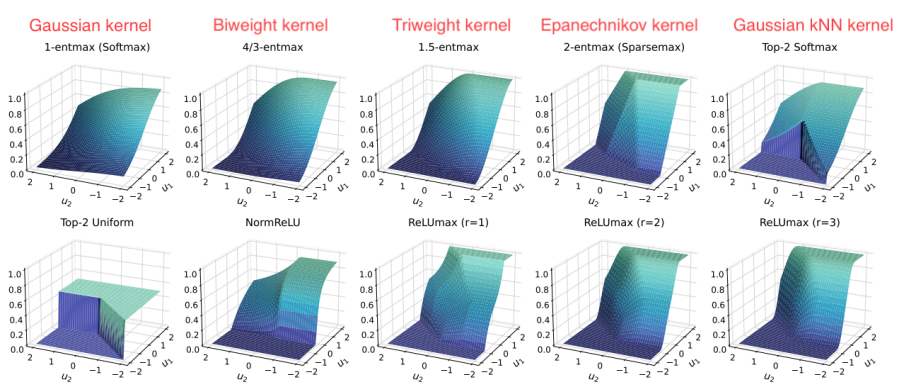
$$\text{entmax}_{\alpha(z)} = \text{argmax}_{p \in \Delta} \left(z^T p + H_{\alpha(p)} \right)$$

Здесь $\Delta = \left\{ p \in \mathbb{R}^n : p_i \geq 0, \sum_i p_i = 1 \right\}$ — вероятностный симплекс, а

$$H_{\alpha(p)} = \frac{1 - \sum_i p_i^\alpha}{\alpha(\alpha - 1)}, \quad \alpha > 1.$$

При $\alpha \to 1$ получаем softmax, при $\alpha = 2$ — sparsemax.





Ядерная интерпретация Normalized ReLU

Ранее мы показали, что softmax attention соответствует гауссовскому ядру. Теперь заменим его на ядро Епанечникова с компактным носителем:

$$K_h(k_i - q) \propto \left[1 - \frac{\|k_i - q\|^2}{h^2}\right]_+$$

Используя $\|k_i\| = \|q\| = 1$, получаем:

$$K_h(k_i - q) \propto \left[1 - \frac{2 - 2k_i^T q}{h^2}\right]_+ = \left[\frac{k_i^T q}{\gamma} + b\right]_+$$

где $\gamma = \frac{h^2}{2}$, $b = 1 - \frac{2}{h^2}$.

Подставим полученное ядро в Nadaraya-Watson estimator:

$$\hat{v}_n(q) = \sum_{i=1}^{n-1} \frac{\left[\frac{k_i^T q}{\gamma} + b\right]_+}{\sum_{j=1}^{n-1}\left[\frac{k_j^T q}{\gamma} + b\right]_+} v_i$$

Получили Normalized ReLU attention: сходства проходят через ReLU, после чего нормируются до суммы, равной единице.

- Разреженность возникает естественно: отрицательные активации обнуляются.
- Проблема: если все активации равны нулю, знаменатель обращается в ноль ($\frac{0}{0}$).

В статье рассматриваются два решения: адаптивный bandwidth и привязка носителя к наиболее похожим ключам.

Автоматический подбор bandwidth приводит к sparsemax

На предыдущем слайде мы получили Normalized ReLU через ядро Епанечникова. Однако нормировка может приводить к неопределённости $\frac{0}{0}$.

Идея: после вынесения общего множителя подобрать bandwidth h так, чтобы сумма масштабированных откликов ядра равнялась единице. Такой подход называется auto-normalization.

Вспомним ядро Епанечникова и выразим его через скалярное произведение нормированных ключей:

$$K_h(k_i - q) \propto \left[1 - \frac{\|k_i - q\|^2}{h^2}\right]_+$$

Введём параметр температуры $\gamma > 0$:

$$\left[1 - \frac{\|k_i - q\|^2}{h^2}\right]_+ \propto \left[\frac{k_i^T q}{\gamma} - \underbrace{\left(\frac{1}{\gamma} - \frac{h^2}{2\gamma}\right)}_{\tau}\right]_+$$

Выберем h так, чтобы:

$$\sum_{i=1}^{n-1} \left[\frac{k_i^T q}{\gamma} - \tau\right]_+ = 1$$

Получаем Sparsemax attention:

$$\hat{v}_n(q) = \sum_{i=1}^{n-1} \text{sparsemax}_i\left(\frac{Kq}{\gamma}\right) v_i$$

- Порог τ зависит от сходства запроса с ключами.
- Bandwidth становится адаптивным: $h = \sqrt{2 - 2\gamma\tau}$.
- Число ненулевых весов определяется автоматически, без фиксированного top-k.

Полиномиальные ядра соответствуют α-entmax

На предыдущем слайде мы показали, что sparsemax соответствует ядерной регрессии с ядром Епанечникова и адаптивным bandwidth.

Авторы обобщают этот результат на целое семейство полиномиальных ядер с компактным носителем:

$$K_h(u) \propto \left[1 - \frac{\|u\|^2}{h^2}\right]_+^r, \quad r > 0$$

Основной теоретический результат: при соответствующем адаптивном выборе bandwidth ядерная регрессия эквивалентна α-entmax attention, если

$$r = \frac{1}{\alpha - 1}, \quad \alpha > 1$$

Частные случаи этого соответствия:

α	r	Ядерная функция
2	1	Epanechnikov
1.5	2	Biweight
$\frac{4}{3}$	3	Triweight

Вывод: sparsemax — частный случай α-entmax. Изменяя степень ядра и адаптивно выбирая bandwidth, можно получить целое семейство разреженных механизмов внимания.

Ограничение носителя вместо изменения ядерной функции

Ранее мы получали разреженность с помощью компактных полиномиальных ядер и адаптивного bandwidth.

Альтернативный подход: сохранить исходное ядро, но ограничить его носитель множеством наиболее похожих ключей (anchored compact kernels).

- Top-k average pooling (uniform-kNN): выбираем k ближайших ключей и усредняем соответствующие значения с одинаковыми весами:

$$\hat{v}(q) = \frac{1}{k} \sum_{i \in N_{k(q)}} v_i$$

- Top-k softmax (truncated Gaussian): используем гауссовские веса только для k ближайших ключей:

$$\hat{v}(q) = \sum_{i \in N_{k(q)}} \frac{\exp\left(\frac{k_i^T q}{\gamma}\right)}{\sum_{j \in N_{k(q)}} \exp\left(\frac{k_j^T q}{\gamma}\right)} v_i$$

Новый метод авторов — r-ReLUmax.

Вместо фиксированного количества ключей носитель привязывается к максимальному сходству:

$$m = \max_j k_j^T q, \quad w_i = \left[b + \frac{k_i^T q - m}{h^2} \right]_+^r, \quad r > 0$$

$K_{r-\mathrm{ReLUmax}}(k_i - q) \propto w_i$, а веса внимания нормируются: $p_i = \frac{w_i}{\sum_j w_j}$.

Здесь $b > 0$ — гиперпараметр, задающий ширину области вокруг максимального сходства.

- Ключ получает ненулевой вес, если $k_i^T q > m - bh^2$.
- Лучший ключ всегда имеет положительный вес, поэтому проблема $\frac{0}{0}$ не возникает.
- Число выбранных ключей зависит от сходств, а не от фиксированного k.

Вывод: top-k ограничивает число ключей, а ReLUmax — допустимое отклонение от максимального сходства.

Архитектура Memory Mosaics

Memory Mosaics (Zhang et al., ICLR 2025) — архитектура, альтернативная классическому трансформеру, основанная на ассоциативной памяти.

Основная идея: предсказание нового токена выполняется через поиск похожих состояний в памяти и извлечение связанных с ними значений.

Блок Memory Mosaics состоит из двух компонентов:

- Contextual Memory — использует предыдущие токены как пары key-value. Реализует Nadaraya-Watson estimator для извлечения информации из текущего контекста.
- Persistent Memory — обучаемые слоты памяти, сохраняющие информацию между последовательностями. Выполняет роль, аналогичную feed-forward сети трансформера.

Особенности архитектуры:

- На текущей позиции query совпадает с key: $q = k_n$.
- Используется causal attention с исключением самосопоставления и сдвигом целевых значений.
- Ассоциативная память позволяет реализовать in-context learning.

Роль в статье: Memory Mosaics служит экспериментальной платформой для сравнения различных ядер и механизмов sparse attention.

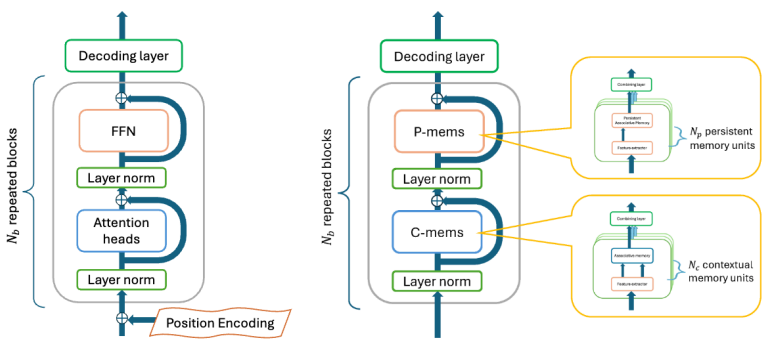


Figure 6: *Left*: Classic GPT2-small transformer. *Right*: GPT2-like Memory Mosaic

Задачи, на которых тестировалась модель

Авторы сравнивают различные механизмы attention в архитектуре Memory Mosaics на трёх группах задач.

1. Language Modeling — моделирование языка

- BabiStories — корпус простых историй на английском языке.
- Предсказание следующего токена по предыдущим.
- Цель: проверить качество языкового моделирования и влияние глубины модели на эффективность разных ядер.

2. In-Context Learning — обучение в контексте

- RegBench — синтетический бенчмарк на основе случайно сгенерированных формальных языков.
- Модель должна выявлять закономерности непосредственно из предъявленной последовательности.
- Цель: проверить способность разных ядер извлекать и использовать информацию из контекста.

3. Length Generalization — обобщение на длинные последовательности

- MQMTAR — ассоциативное воспроизведение информации.
- Sequence Reversal — разворот последовательности.
- Sequence Sorting — сортировка последовательности.
- Цель: проверить, сохраняется ли качество на последовательностях длиннее обучающих.

Главный вопрос экспериментов: помогают ли компактные ядра и разреженное внимание лучше использовать контекст и обобщать на новые условия?

Результаты Language Modeling

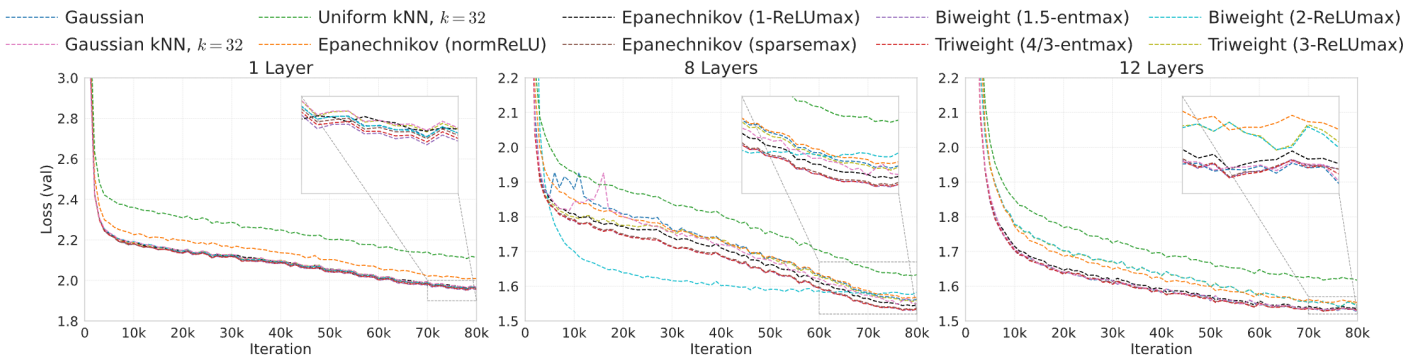


Figure 2: **Validation loss of Memory Mosaic models with different kernels.** Comparison on the BabiStories dataset for varying model depths. The horizontal axis shows the number of training iterations.

Интерпретация результатов:

- 1 слой: ядра с компактным носителем и адаптивным bandwidth превосходят Gaussian (softmax) и top-k по validation loss.
- 8 слоёв: преимущество адаптивных ядер сохраняется. Sparsemax и α-entmax показывают лучшие результаты, чем плотное внимание и top-k.
- 12 слоёв: различия уменьшаются. Большинство методов демонстрирует близкие результаты, кроме NormReLU и Uniform kNN, которые уступают остальным.

Вывод: адаптивная нормировка особенно эффективна в неглубоких моделях. С увеличением глубины преимущество разреженного внимания уменьшается.

Результаты In-Context Learning

Table 1: **Performance comparison across varying training set sizes in the Regbench in-context learning benchmark.** Accuracy (↑) measures the correct prediction rate, and TVD (↓) measures divergence from the target distribution. We **bold** the best performing models.

Method	1k		2.5k		5k		10k		20k		40k	
	Acc ↑	TVD ↓	Acc ↑	TVD ↓	Acc ↑	TVD ↓	Acc ↑	TVD ↓	Acc ↑	TVD ↓	Acc ↑	TVD ↓
RetNet	51.3	67.4	50.5	63.9	81.5	33.3	88.4	25.7	90.8	22.5	92.6	18.9
H3	51.5	65.1	58.0	57.0	70.9	47.4	76.9	41.4	80.7	36.3	87.7	25.4
Gla	51.8	65.3	51.1	62.2	82.1	31.0	90.2	24.5	91.2	21.2	92.9	19.0
S4	52.1	62.8	51.7	62.2	54.5	59.0	63.3	52.9	72.1	45.8	82.5	33.8
Mamba	69.4	51.0	79.4	41.0	88.7	30.6	90.5	28.4	91.3	27.0	91.5	27.0
Transformer	54.5	62.6	92.4	20.5	92.7	18.9	94.8	15.7	95.1	15.0	95.5	13.9
Memory Mosaics:												
Gaussian	93.9	34.1	93.7	20.8	94.6	17.7	95.0	15.9	**95.4**	15.3	95.4	14.2
Gaussian kNN, $k = 16$	93.4	33.3	93.7	19.8	94.4	17.2	94.8	16.0	94.8	16.0	95.0	14.9
Uniform kNN, $k = 16$	84.6	41.1	85.6	28.3	86.1	27.2	86.2	26.4	86.5	26.4	86.4	26.4
Epanechnikov (normReLU)	84.3	43.6	87.0	29.6	90.1	24.6	91.6	21.7	93.1	19.4	93.3	18.6
Epanechnikov (1-ReLUmax)	93.8	36.8	94.2	20.4	94.3	17.8	94.9	**15.7**	**95.4**	**15.0**	95.4	14.1
Epanechnikov (sparsemax)	**94.2**	**32.5**	94.0	19.3	94.4	17.1	94.9	15.9	95.0	**15.0**	95.4	14.0
Biweight (2-ReLUmax)	93.8	36.4	94.4	20.1	94.6	17.2	95.1	16.1	95.3	15.1	95.1	14.8
Biweight (1.5-entmax)	93.8	33.0	**94.3**	19.2	94.7	16.6	**95.2**	**15.7**	95.0	15.8	**95.5**	**13.8**
Triweight (3-ReLUmax)	93.9	36.6	94.2	20.5	94.7	16.9	94.8	16.4	95.1	15.4	95.3	14.3
Triweight (4/3-entmax)	93.9	32.9	**94.3**	**19.0**	**94.8**	**16.2**	94.9	16.1	95.1	15.3	95.3	14.4

Интерпретация результатов:

- При 1k обучающих примеров: Memory Mosaics значительно превосходит стандартный Transformer по accuracy. При 2.5k–5k разрыв существенно меньше. Преимущество наблюдается и у Gaussian Memory Mosaics.
- Sparsemax, Biweight и Triweight показывают высокую accuracy и низкую TVD. С увеличением обучающей выборки различия между лучшими методами уменьшаются.
- NormReLU и Uniform kNN заметно уступают адаптивным методам. Простого обнуления весов само по себе не гарантирует высокого качества.

Вывод: качество зависит от способа выбора и взвешивания активных ключей. Сама разреженность не гарантирует улучшения; компактные ядра дают преимущества в ряде режимов с малым объёмом данных.

Результаты Length Generalization

Table 2: Exact match accuracy on representative synthetic tasks. In-distribution results ($n = 64$) and out-of-distribution performance at increasing sequence lengths are reported. Values show the mean across three seeds, with the maximum across seeds indicated in superscript. Best average performance is in **bold** and overall maximum performance is underlined. L is the number of layers.

	MQMTAR ($L = 4$)							Reverse ($L = 4$)		Sort ($L = 2$)		
Kernel	1×	2×	4×	8×	16×	32×	64×	1×	1.5×	1×	2×	4×
Transformer	**1.00**$^{(1.00)}$	**1.00**$^{(1.00)}$	0.99$^{(0.99)}$	0.09$^{(0.15)}$	0.01$^{(0.01)}$	0.00$^{(0.00)}$	0.00$^{(0.00)}$	**1.00**$^{(1.00)}$	**0.52**$^{(0.62)}$	**1.00**$^{(1.00)}$	0.00$^{(0.00)}$	0.00$^{(0.00)}$
Gaussian	0.99$^{(1.00)}$	0.96$^{(1.00)}$	0.93$^{(0.99)}$	0.85$^{(0.97)}$	0.64$^{(0.81)}$	0.14$^{(0.20)}$	0.00$^{(0.00)}$	**1.00**$^{(1.00)}$	0.14$^{(0.38)}$	**1.00**$^{(1.00)}$	0.06$^{(0.17)}$	0.00$^{(0.00)}$
Gaussian kNN, $k = 32$	0.99$^{(1.00)}$	0.97$^{(1.00)}$	0.93$^{(0.99)}$	0.84$^{(0.96)}$	0.68$^{(0.85)}$	0.27$^{(0.36)}$	0.05$^{(0.05)}$	0.99$^{(1.00)}$	0.00$^{(0.00)}$	**1.00**$^{(1.00)}$	0.80$^{(0.91)}$	0.01$^{(0.02)}$
Uniform kNN, $k = 32$	0.04$^{(0.08)}$	0.00$^{(0.00)}$	0.00$^{(0.00)}$	0.00$^{(0.00)}$	0.00$^{(0.00)}$	0.00$^{(0.00)}$	0.00$^{(0.00)}$	0.42$^{(0.66)}$	0.00$^{(0.00)}$	**1.00**$^{(1.00)}$	0.00$^{(0.00)}$	0.00$^{(0.00)}$
Epanechnikov (normReLU)	**1.00**$^{(1.00)}$	**1.00**$^{(1.00)}$	0.89$^{(1.00)}$	0.66$^{(1.00)}$	0.44$^{(1.00)}$	0.33$^{(0.98)}$	**0.31**$^{(0.94)}$	**1.00**$^{(1.00)}$	0.19$^{(0.56)}$	**1.00**$^{(1.00)}$	0.05$^{(0.14)}$	0.00$^{(0.00)}$
Epanechnikov (1-ReLUmax)	**1.00**$^{(1.00)}$	**1.00**$^{(1.00)}$	0.99$^{(1.00)}$	0.92$^{(0.96)}$	0.74$^{(0.87)}$	0.42$^{(0.59)}$	0.12$^{(0.18)}$	**1.00**$^{(1.00)}$	0.23$^{(0.68)}$	**1.00**$^{(1.00)}$	0.80$^{(1.00)}$	0.30$^{(0.91)}$
Epanechnikov (sparsemax)	**1.00**$^{(1.00)}$	**1.00**$^{(1.00)}$	**1.00**$^{(1.00)}$	**0.98**$^{(1.00)}$	0.86$^{(0.98)}$	**0.63**$^{(0.84)}$	**0.31**$^{(0.51)}$	**1.00**$^{(1.00)}$	0.03$^{(0.06)}$	**1.00**$^{(1.00)}$	0.92$^{(1.00)}$	0.29$^{(0.79)}$
Biweight (2-ReLUmax)	0.99$^{(1.00)}$	0.97$^{(1.00)}$	0.93$^{(1.00)}$	0.85$^{(0.96)}$	0.69$^{(0.87)}$	0.27$^{(0.43)}$	0.04$^{(0.06)}$	**1.00**$^{(1.00)}$	0.26$^{(0.74)}$	**1.00**$^{(1.00)}$	0.93$^{(1.00)}$	**0.34**$^{(0.74)}$
Biweight (1.5-entmax)	**1.00**$^{(1.00)}$	**1.00**$^{(1.00)}$	0.99$^{(1.00)}$	0.95$^{(0.99)}$	0.75$^{(0.93)}$	0.51$^{(0.74)}$	0.23$^{(0.42)}$	**1.00**$^{(1.00)}$	0.05$^{(0.06)}$	**1.00**$^{(1.00)}$	0.39$^{(1.00)}$	0.26$^{(0.77)}$
Triweight (3-ReLUmax)	0.99$^{(1.00)}$	0.97$^{(1.00)}$	0.93$^{(0.99)}$	0.85$^{(0.96)}$	0.71$^{(0.87)}$	0.37$^{(0.58)}$	0.08$^{(0.18)}$	**1.00**$^{(1.00)}$	0.29$^{(0.88)}$	**1.00**$^{(1.00)}$	0.95$^{(0.97)}$	0.30$^{(0.62)}$
Triweight (4/3-entmax)	**1.00**$^{(1.00)}$	**1.00**$^{(1.00)}$	0.99$^{(1.00)}$	0.97$^{(0.98)}$	**0.89**$^{(0.95)}$	0.61$^{(0.76)}$	0.25$^{(0.45)}$	**1.00**$^{(1.00)}$	0.26$^{(0.70)}$	**1.00**$^{(1.00)}$	**0.98**$^{(0.99)}$	0.07$^{(0.11)}$

Интерпретация результатов:

- Gaussian (softmax) — хорошо работает на обучающей длине, но заметно теряет качество при увеличении последовательности.
- Uniform kNN — плохо экстраполирует на длинные последовательности. Одного фиксированного top-k недостаточно для устойчивого обобщения.
- Gaussian kNN — ограничение внимания ближайшими ключами существенно замедляет деградацию на задаче ассоциативного воспроизведения.
- MQMTAR: sparsemax и α-entmax заметно улучшают обобщение на длинные последовательности.
- Reverse и Sort: результаты сильно зависят от метода и тестовой длины. Даже лучшие методы теряют качество; устойчивое обобщение на MQMTAR не переносится на все задачи.

Вывод: адаптивная разреженность может улучшать обобщение на длинные последовательности, но степень улучшения зависит от задачи.

Выводы

1. Единая интерпретация sparse attention

- Авторы показали, что различные механизмы разреженного внимания можно представить как ядерную регрессию с компактным носителем.
- Sparsemax соответствует ядру Епанечникова с адаптивным bandwidth, а α-entmax — семейству полиномиальных ядер.

2. Способы получения разреженности

- Adaptive bandwidth — носитель ядра подстраивается под распределение сходств ключей с запросом.
- Anchored kernels — носитель определяется ближайшими ключами или максимальным сходством.
- Авторы предложили новый механизм r-ReLUmax, обеспечивающий разреженность без фиксированного top-k.

3. Экспериментальные результаты

- Адаптивные компактные ядра часто превосходят стандартный softmax attention в Memory Mosaics.
- Наиболее заметные преимущества наблюдаются в задачах in-context learning и обобщения на длинные последовательности.
- Однако эффективность зависит от задачи, глубины модели и способа определения носителя.

Ограничения: эксперименты проведены на сравнительно небольших моделях и специализированных задачах. Необходима проверка результатов на более крупных трансформерах и реальных языковых корпусах.